

Privacy-Preserving Collaborative IDS for Internet-of-Vehicle Networks

Thanking,
Dr. Durga
Dr. Uma
Dr. Anusuya
Dr. Praveen



#TITLE OF THE PROJECT , 28-08-2026

Recap from Last Meeting

- ChebyKAN adopted as the IDS trunk — 44,164 parameters, 14× smaller than the MamKANformer trunk
- Layer-wise homomorphic federated aggregation demonstrated across all four datasets
- Committed to CKKS (FheFL) so the server never sees an individual plaintext update
- Review action item: present the mathematical foundations of CKKS — addressed in this deck



Contents

- Mathematical foundations of CKKS
- Baseline: FHE vs No-FHE — the control established before model quantization
- System validation — proactive data-engineering validation
- V2X communication protocol — FHE with AES / ECDH
- Multi-key CKKS



Where We Stand

- Threat model, security specification, library evaluation, packing & encoding module, NTT reference implementation
- Automated tests guarding data integrity, cryptographic correctness, and the privacy invariant — all passing
- Real CKKS now replaces the placeholder throughout the federated loop
- The homomorphic aggregator is implemented now



Mathematical Foundations — The CKKS Scheme (worked example)

CKKS Homomorphic Encryption in simplified form

Secret Key $\rightarrow S = [4, 9]$

Modulus $q = 100003$

Scaling Factor $\Delta = 1000$

weights in Plaintext

$C_1 = 0.10$

$C_2 = 0.12$

$C_3 = 0.09$

$$\Rightarrow b = a \cdot s + e + \Delta \cdot PT \pmod{q}$$

where

$a \rightarrow$ random vector

$e \rightarrow$ random noise

a is public and different

Drawback,

All clients must have same secret key

$S = [4, 9]$ • A single compromised client can lead to total leak.



Mathematical Foundations — Homomorphic Aggregation (worked example)

Encryption

$$\Rightarrow b = a \cdot s + e + \Delta \cdot PT \pmod{q}$$

client-1 (PT=0.10)

$$a_1 = [37, 52]$$

$$e_1 = 5$$

$$b_1 = (37 \times 4 + 52 \times 9) + 5 + 100$$

$$b_1 = 721$$

$$CT_1 = (a_1, b_1)$$

$$CT_1 = ([37, 52], 721)$$

client-2 (PT=0.12)

$$a_2 = [21, 66]$$

$$e_2 = 3$$

$$b_2 = 801$$

$$CT_2 = ([21, 66], 801)$$

client-3 (PT=0.09)

$$a_3 = [58, 14]$$

$$e_3 = 4$$

$$b_3 = 452$$

$$CT_3 = ([58, 14], 452)$$



Mathematical Foundations — Homomorphic Aggregation (worked example)

Aggregation of Encrypted weights on server side

Let us now take Averaging

$$a_sum = a_1 + a_2 + a_3 \quad b_sum = b_1 + b_2 + b_3$$

$$a_sum = [116, 132] \quad b_sum = 1974$$

Averages (3-clients so divide by 3)

$$a_{avg} = [38.66, 44] ; b_{avg} = 658$$



Mathematical Foundations — Homomorphic Aggregation (worked example)

Broadcast $[a_{avg}, b_{avg}]$ to clients

Decryption: [on client side with $s = [4, 9]$]

$$i = a_{avg} \cdot s = 38.66 \times 4 + 44 \times 9 = 550.64$$

$$r = b_{avg} - i = 658 - 550.64 = 107.36$$

Divide r by scaling factor Δ

$$Res = r/\Delta = 107.36/1000 = 0.10736$$

Final Decrypted aggregated value = 0.10736 //

$$\text{True Avg of P.T weights} = (0.1 + 0.12 + 0.09)/3 = 0.1033$$

$$\text{Error} \approx 0.004$$

Smaller error is not an issue, it is due to lower modulo and noise.

In real cases modulo will be higher thus error becomes negligible.



Mathematical Foundations — Homomorphic Aggregation (Real CKKS case)

Real case

Full layer weights are transformed as polynomials

$$[a, b, c] \rightarrow (a + bx + cx^2) \cdot \Delta$$

⇒ Secret key

will be a

Polynomial Ring

$$s(x) \in R_q$$

$$R_q = \mathbb{Z}_q[x] / x^{N+1}$$

⇒ Random vector

$a(x) \rightarrow$ Random Polynomial

⇒ Evaluation key (error)

$e(x) \rightarrow$ Polynomial with lower degree

$$\therefore b(x) = a(x) \cdot s(x) + e(x)$$



Mathematical Foundations — Homomorphic Aggregation (Real CKKS Case)

⇒ Public key

$$(a, b) = (a(x), b(x))$$

For Example Let

$$\text{P.T weights } m(x) = 288 + 325x + 471x^2$$

Random $u(x)$ and Noise $e_1(x), e_2(x)$

C.T components

$$C_0 = b \cdot u + e_1 + m$$

$$C_1 = a \cdot u + e_2$$

$$CT = (C_0, C_1)$$

Decryption

$$m'(x) = C_0 + C_1 \cdot s$$

This value is found and divided by Δ



Mathematical Foundations — Noise, Depth, and Why It Shapes the Design

- Every ciphertext carries growing noise; every ciphertext $\times$ ciphertext multiply consumes one prime ("level") from the modulus chain via rescaling
- A chain of L primes supports $L - 2$ multiplications — the depth budget is the binding design constraint
- Security is set by ring dimension N vs total modulus bits: at $N = 8192$, 128-bit security allows ≤ 218 bits $\rightarrow$ 2 multiplicative levels
- Homomorphic addition is about 490 $\times$ cheaper than ciphertext multiplication (measured)
- Consequence: the FheFL protocol is engineered to need as few ciphertext $\times$ ciphertext multiplies as possible



Baseline: FHE vs No-FHE — Setup

- Purpose: the mandatory control before model quantization — measure what encryption costs before adding a second approximation
- Real TenSEAL CKKS on one arm, exact plaintext on the other, everything else held identical
- 3 seeds · 10 simulated vehicles · 5 federated rounds · Dirichlet $\alpha = 0.3$ · 70% client sampling
- The two backends are asserted bitwise-identical on the aggregation path (test_backend_equivalence)



Baseline: FHE vs No-FHE — Results

Dataset	CKKS acc	Plaintext acc	Δ	Verdict
CAN-VTC	1.0000	1.0000	+0.0000	within seed noise
Car-Hacking	0.9977	0.9981	-0.0004	within seed noise
CICIDS-2017	0.9555	0.9542	+0.0012	within seed noise
VeReMi	0.4839	0.4851	-0.0012	within seed noise

- Finding: encryption costs no accuracy — the CKKS/plaintext gap is smaller than seed-to-seed variance on every dataset
- CAN-VTC = 100% is flood detection, not generalisation: the DoS attack floods one novel CAN ID (51% of traffic), separable by a single feature at 13σ . Split verified clean — the task is simply easy, not the earlier filename-label defect.
- Car-Hacking is the CAN benchmark we stand behind: 5 real classes, best single feature only 0.94, injection density 12–24%
- VeReMi is a near-chance lower bound: the cleaned export dropped sender IDs, so the misbehaviour signal cannot be expressed — re-acquisition required



What This Baseline Establishes

- Real CKKS now replaces the previous placeholder (SimulatedCKKSVector), so the FHE-vs-plaintext comparison is a controlled experiment for the first time
- Accuracy is encryption-invariant → any accuracy change under INT8 quantization is therefore attributable to quantization alone, not to encryption
- Measured cryptographic cost per client update (44,164 parameters): encrypt ~42 ms · encrypted distance ~118–230 ms · homomorphic aggregation ~5–52 ms · ciphertext on the wire 2.47 MB



System Validation — Proactive Data-Engineering Validation

- Before building the multi-key layer, we ran a full validation pass over the existing pipeline: check the foundation before adding a floor
- Seven issues identified and all fixed, each now covered by a regression test:
 - Ground-truth labels now per-message (HCRL R/T flag), not per-file
 - Train/test split now group-aware by contiguous block — no window leakage
 - FHE vs No-FHE now a controlled comparison (real CKKS, bitwise-checked)
 - Server no longer decrypts individual client updates — FheFL scoring replaces Multi-Krum
 - Transport security (AES / ECDH) restored
 - VeReMi formulation diagnosed (needs sender IDs); documentation drift corrected



System Validation — Effect on Results and the Privacy Invariant

- Headline accuracy figures were revised on the corrected pipeline: Car-Hacking 0.74 → 0.998 (fixed labels made the task well-posed); CICIDS 0.982 → 0.955 (leakage removed); CAN-VTC and VeReMi not comparable across formulations
- These movements are the experiment starting to measure the right thing
- Privacy invariant, machine-checked every round: individual client updates decrypted = 0 on all four datasets across 5 rounds each; only the aggregate is decrypted



V2X Communication Protocol — FHE with AES / ECDH

- Two complementary layers, not alternatives:
 - FHE → payload confidentiality from the aggregation server
 - AES-256-GCM / ECDH-P256 / Ed25519 → authentication, integrity, replay resistance, Sybil resistance on the radio link
- FHE provides none of the second list; removing the channel in last time traded a working control for a slogan
- Status: the transport channel is restored and integrated; mutual-auth handshake tested (test_secure_channel)



V2X — The Bandwidth Problem

- One 44k-parameter update = 2.47 MB encrypted (~915 DSRC frames); a 50-vehicle round $\approx$ 180 MB — does not fit a V2X budget
- Delivered: Optimized Packing & Encoding module — 29.6% lossless reduction via level-dropped transmission, verified at zero accuracy cost
- Further levers: seed compression (~50%, needs OpenFHE) · sparsification (needs an accuracy study) · hierarchical RSU pre-aggregation
- Hierarchical aggregation is likely mandatory — an architecture decision to take early



Why Multi-Key CKKS

- **Federated Learning** keeps raw data on IoT devices, but individual model updates can still reveal private information.
- **Homomorphic Encryption (HE)** allows the server to perform computations on encrypted model updates.
- **Single-key HE** can create privacy risks because the same secret key may be shared/accessible across participants.
- **MK-CKKS** gives each device its **own secret key**, improving key isolation.
- However, the paper identifies a **privacy leakage problem during individual decryption** in MK-CKKS.
- **xMK-CKKS** is proposed to protect individual model updates while still allowing secure aggregation.
- **Source:** [\[2104.06824\] Privacy-preserving Federated Learning based on Multi-key Homomorphic Encryption](#)



How xMK-CKKS Works

- Each device generates its **own secret key S_i** and corresponding **public key P_i**
- The public keys are combined into an **aggregated public key**: $\tilde{P} = P_1 + P_2 + \dots + P_N$
- Devices perform **local training** and encrypt their model updates under the **aggregated public key $\tilde{P}$** — not under their own P_i
- The server performs **homomorphic addition** on the ciphertexts without seeing individual model updates.
- The server obtains an **encrypted aggregate** and returns the aggregated ciphertext component to the devices.
- Each device uses its **secret key S_i** and the **aggregated ciphertext** to generate a **decryption share D_i** , then sends it to the server



How xMK-CKKS Works

Step 1:-

Client 1 client 2 client 3
 SK_1 SK_2 SK_3

Each client creates
 a secret key SK_i

Step 2:-

Each client
 creates its
 own public key

here
 $a \rightarrow$ Common public
 parameter

$e_i \rightarrow$ error / noise term

$$b_i = -SK_i \cdot a + e_i$$

Step 3:-

Public keys are combined

$$\tilde{b} = b_1 + b_2 + b_3 \quad \tilde{b} = \sum_{i=1}^N b_i$$

$\tilde{b} \rightarrow$ Aggregated public key

Step 4:-

$\rightarrow$ local client training
 $\rightarrow$ Encrypt weights using $\tilde{b}$
 (Aggregated public key)

How xMK-CKKS Works

Step 5:-

→ Encrypted weights go to the server

→ server adds the Encrypted weights (aggregation)

$$C_{sum} = CT_1 + CT_2 + CT_3 \quad \rightarrow C_{sum} \text{ has two components}$$

$$C_{sum} = (C_{sum,0}, C_{sum,1}) \quad \rightarrow C_{sum} \text{ is Broadcasted to the clients}$$

Step 6:-

Each client creates its own Decryption share using

→ SK_i , $C_{sum,1}$ and additional noise

$$D_i = SK_i \cdot C_{sum,1} + C_i^* \quad D_i \rightarrow \text{Decryption share}$$

→ clients send their Decryption shares to the server

How xMK-CKKS Works

Step 7:-

→ Server combines all the Decryption shares sent by clients along with $C_{sum,0}$

$$C_{sum,0} + D_1 + D_2 \cdots D_N \approx W_1 + W_2 \cdots W_N$$

The server has decrypted aggregated value.

→ Then average and broadcast to client to update the model



Secure Aggregation & Benefits

- The server combines the **decryption shares** with the aggregated ciphertext to recover **only the sum of the model updates**
- The server computes the new global model using **Federated Averaging**: $W_{t+1} = (1/N) \sum_{i=1}^N W_i$
- Individual updates $W_1, W_2, \dots, W_N$ remain protected from the server under the paper's threat model
- xMK-CKKS is designed to resist **collusion between the server and $k < N - 1$ participating devices**



Status and the upcoming works to be done

- Ciphertext serialization & comms protocol: **partial** — authenticated transport channel complete; serialization + fragmentation spec open
- Model quantization: **not started** — the encryption-neutral accuracy baseline is now established as its control
- Multi-key CKKS: **xMK-CKKS selected** — the homomorphic arithmetic is real CKKS today; the key management is still an algebraic model
- **Future plan:**
 - OpenFHE migration — makes xMK-CKKS real rather than modelled
 - Per-round aggregated public key $\tilde{P}$ over the participant set — removes the dominant dropout case at no extra cost
 - Slot-batched distance decryption — keeps a round at 2 distributed decryptions instead of $N + 1$
 - Low-magnitude persistent-poisoning experiment — the known blind spot in the scoring function
 - Scale from 10 to 50 clients; re-acquire raw VeReMi with sender IDs

